NETLENS — PHASE 1 REFERENCE

Field Notes

A working file-by-file map of what Phase 1 actually built, why each piece exists, how the pieces fit together, and the exact steps to bring the live demo back up on demand.

Capture → Decode → Flow → Store → Display • Python 3.14, Scapy, Rich, SQLite, pytest • Confirmed adapter: Local Area Connection* 4

scapy — wire access only | struct — manual header parsing | sqlite3 — persistence | rich — terminal UI | pytest — 19 tests

What Phase 1 actually is

Not the full two-week Round 1 build — a tighter, three-day slice (roughly a quarter of the total project) built to prove the foundation works before the rest is built on top of it.

NetLens's job, eventually, is to watch a phone's traffic and flag suspicious behaviour. That's impossible without first being able to *see* the traffic, understand it, and remember it — so Phase 1 builds exactly that foundation, end to end, and nothing past it.

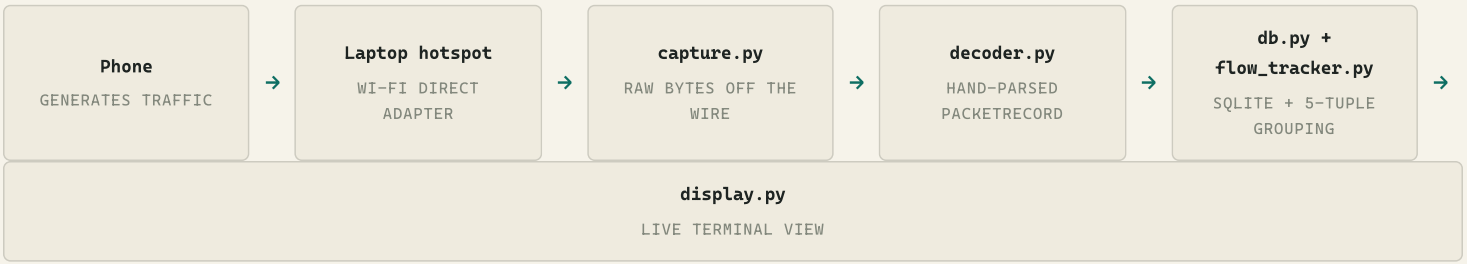
- ✓ **Capture** — the laptop becomes a Wi-Fi hotspot and captures the phone's live traffic in real time.
- ✓ **Decoder** — Ethernet, IP, TCP/UDP headers parsed by hand, byte by byte. ARP and DNS identified with key fields pulled out.
- ✓ **Flow tracker** — packets grouped into conversations by the 5-tuple (source IP/port, destination IP/port, protocol).
- ✓ **Storage** — every decoded packet and every flow persisted to SQLite.
- ✓ **Interface** — a terminal view showing a live packet feed and a flow table.

Deliberately **not** in Phase 1: resolving IPs to organisation names, learning a traffic baseline, and the three detection rules. Those come immediately after, once this foundation is verified working — they're sequenced later, not skipped.

> **WHY THIS SLICE SPECIFICALLY**
>
> It's the one everything else depends on — detection can't exist without decoded, stored packets. It can't be faked by importing a library, since hand-parsing headers byte-by-byte is exactly the part a shortcut can't produce. And it's demoable end to end on its own: live phone traffic in, correctly parsed and grouped data out.

How the pieces connect

One direction of data flow, five modules, each owned independently and wired together only in `main.py`.



`shared/packet_record.py` is the contract that makes this possible: every module agrees on the exact shape of a decoded packet before any of them touch the others' code, so capture, decoding, and storage could all be built in parallel without collisions.

Every file, what it does, and why

Ordered the way data actually flows through them: contract first, then capture, decode, storage, wiring, and finally the tests that hold it all honest.

The shared contract

`shared/packet_record.py` — DATA CONTRACT

`DATACLASS`

Defines `PacketRecord` — the one shape every decoded packet must take, no matter where it came from (live capture, a test fixture, a replayed file). Capture produces raw bytes, the decoder turns those bytes into a `PacketRecord`, and storage only ever consumes `PacketRecord`s. No module invents its own shape.

Holds capture metadata (timestamp, length), Ethernet fields (MACs, EtherType), the network/transport 5-tuple, IPv4 extras (TTL, total length), TCP extras (flags, sequence and ack numbers), ARP extras (opcode, sender/target MAC), and DNS extras (query name, type, answers) — almost everything is `Optional`, since a given packet only fills in the fields relevant to its own protocol.

Capture

`capture/capture.py` — GETS BYTES OFF THE WIRE

`SCAPY.SNIFF`

Its only job is handing back `(raw_bytes, timestamp)` for every frame seen on the chosen interface — it does **not** parse anything. Scapy is used here purely as the thing that talks to Npcap and pulls frames off the NIC driver; every frame is re-serialised with `bytes(pkt)` before being handed onward, so what the decoder sees is the untouched wire data, not a scapy object.

`capture_packets(iface, count)` — generator yielding raw frames as they arrive, via a background thread and a queue so the main loop never blocks on scapy's own event loop.

Requires an elevated (Administrator) terminal and Npcap installed in WinPcap-compatible mode — raw NIC access is privileged on Windows. Missing either surfaces as a `PermissionError` or `OSError`, both caught and explained in `main.py` rather than left as a raw traceback.

Decoder

`decoder/decoder.py` — BYTES → STRUCTURED RECORD

STRUCT.UNPACK TCP UDP ARP DNS ICMP

The heart of the "prove it wasn't faked" requirement. Every header field is pulled out with `struct.unpack` or direct byte slicing at a known offset — scapy's dissection is never used to do the actual parsing (it's only used elsewhere, to *build* test fixtures). This file is a working implementation of RFC 894 (Ethernet), RFC 791 (IPv4), RFC 793 (TCP), RFC 768 (UDP), RFC 826 (ARP), and RFC 1035 (DNS).

`decode(raw_bytes, timestamp)` — the entry point; returns one populated `PacketRecord`.

`_parse_ethernet` — 14-byte fixed header, with 802.1Q VLAN tag handling.

`_parse_ipv4` — variable-length header via IHL, TTL, protocol number, fragment detection.

`_parse_tcp` / **`_parse_udp`** — ports, sequence/ack numbers, flag bits (SYN/ACK/FIN/etc.).

`_parse_arp` — fixed 28-byte IPv4-over-Ethernet ARP packet.

`_parse_dns_name` / **`_parse_dns`** — label-by-label name parsing, including following compression pointers (RFC 1035 §4.1.4).

Every parser is defensive about length — a header too short to safely read is reported as `OTHER` / truncated instead of raising, so one malformed frame from real traffic never takes the whole capture down.

Storage — flows, database, display

`storage/flow_tracker.py` — GROUPS PACKETS INTO CONVERSATIONS

Groups a list of `PacketRecord`s into flows keyed by the 5-tuple `(src_ip, src_port, dst_ip, dst_port, protocol)`. A request and its reply land in *different* flows, since their source/destination are swapped — a deliberate Phase 1 simplification; merging both directions into one bidirectional flow is scoped for later.

`group_into_flows(records)` — returns a dict of 5-tuple → `FlowStats`.

`FlowStats` — packet count, byte count, first-seen / last-seen timestamps per flow.

storage/db.py — PERSISTENCE

SQLITE3

Owns two tables. `packets` mirrors every `PacketRecord` field 1:1, so nothing is lost on a store/retrieve round trip — the one wrinkle is `dns_answers`, a Python list, which is stored as a comma-joined string and split back out on read. `flows` is a snapshot keyed on the 5-tuple, holding running packet/byte counts and first/last-seen timestamps; it's re-derived from every packet seen so far and **upserted** on each refresh, rather than appended to, so it always reflects current totals.

`get_connection(db_path)` — opens (creating if needed) `storage/netlens.db`.

`store_packet` / `get_all_packets`

`store_flows` / `get_all_flows`

storage/display.py — TERMINAL OUTPUT

RICH

`print_packet_row(record)` — one colour-coded line per packet as it's decoded.

`print_flow_table(flows)` — the bordered "Flows" summary table, refreshed periodically.

Protocol colours are consistent everywhere in the tool: TCP cyan, UDP magenta, ARP yellow, ICMP green, everything else grey.

storage/sample_records.py — FIXTURE DATA

Six hardcoded `PacketRecord`s (a TCP handshake plus a data packet, a DNS query and response, one ARP request) — not real captured traffic, just enough variety to sanity-check storage, flow grouping, and display before live capture was wired in. Now doubles as the fixture the automated test suite runs against.

Decoder test fixtures

decoder/test_data/generate_samples.py — BUILDS KNOWN-GOOD RAW FRAMES

The one place scapy is used to *build* rather than parse: it constructs real TCP/UDP/ARP/DNS frames (computing checksums/lengths for us) and writes the raw bytes to `.bin` files. `decode()` never sees the scapy object — only the serialized bytes, exactly as `capture.py` would hand them over. This is what makes the decoder tests possible without a live capture running.

`decoder/test_data/run_samples.py` — MANUAL EYEBALL CHECK

Loads each `.bin` fixture, decodes it, and prints every non-empty field — a quick manual sanity pass separate from the pytest suite.

Integration

`main.py` — WIRES EVERYTHING TOGETHER

The only file that knows about all four other modules. It sequences `capture_packets() → decode() → store_packet() → print_packet_row()` for every frame, and every 50 packets (or 10 seconds, whichever comes first) it re-derives flows from everything stored so far, persists them with `store_flows()`, and prints the flow table. Also owns the `--iface` and `--list-interfaces` command-line flags, and turns the two most common failure modes — no Administrator privileges, no Npcap — into a plain-English message instead of a stack trace.

Automated tests

`tests/` — 19 TESTS, 3 FILES

`test_decoder.py` — decodes each `.bin` fixture and checks the extracted fields match what the frame was built with (TCP flags/ports, UDP ports, ARP opcode/addresses, DNS query name and answers), plus two failure-mode tests (truncated frame, unrecognised EtherType).

`test_flow_tracker.py` — confirms 5-tuple grouping, same-direction packets accumulating into one flow, reverse direction counted separately, and ARP records grouping correctly without ports.

`test_storage.py` — round-trips packets and flows through a throwaway SQLite database, including the DNS-answers list and the flows table's upsert behaviour.

Run any time with `pytest tests/ -v` — no live capture or Administrator privileges required.

Everything else

`requirements.txt` — 3 DEPENDENCIES, 3 JOBS

`scapy` — wire access only (never used for parsing). `rich` — terminal rendering. `pytest` — the test runner. Nothing else.

The flow table, explained

A packet feed shows you every frame. A flow table answers a different question: *who was talking to whom, and how much?*

A single web page load can be dozens of individual packets — a DNS lookup, a TCP handshake, several data packets, an ACK for each. Scrolling through them one at a time tells you what happened at the wire level, but not the shape of the conversation. Grouping by the 5-tuple `(source IP, source port, destination IP, destination port, protocol)` collapses all of those into one row: *this device talked to that device over this protocol, N packets, this many bytes, from this time to that time.* That conversation-level view is what later detection rules (baseline learning, anomaly flags) will actually reason over — nobody baselines individual packets, they baseline behaviour per flow.

Here's an actual flow table captured mid-demo, annotated:

PROTOCOL	SOURCE	DESTINATION	PACKETS	BYTES
ARP	0.0.0.0	169.254.217.79	1	42
OTHER	–	224.0.0.22	25	2214
UDP	192.168.137.1:137	192.168.137.255:137	9	990
UDP	192.168.137.1:68	255.255.255.255:67	1	342
ARP	0.0.0.0	192.168.137.1	1	42

- **ARP 0.0.0.0 → 192.168.137.1** — a device (the phone) probing "is anyone using this address?" the moment it joins the network. The very first packet a device sends, before it has an IP.
- **UDP :68 → :67** — DHCP. The laptop, acting as the hotspot's address server, handing the phone an IP lease.
- **UDP :137 → :137** — NetBIOS name broadcast. The laptop announcing its own name on the local network — Windows housekeeping, unrelated to the phone's browsing.
- **OTHER → 224.0.0.22** — IGMP multicast group membership reports. More background noise, not application traffic.

None of this is your actual browsing yet — there's no TCP, no port 443, no DNS to a real server. This table is what the *first* ~50 packets look like: a device joining a network, before it starts talking to anything interesting. Real browsing shows up as TCP rows with `SYN` / `SYN,ACK` / `ACK` flags to real destination IPs, and UDP rows to your DNS server on port 53.

Demo runbook

Do this every time you want to bring the live demo back up.

1 LAPTOP

Turn on the mobile hotspot

Settings → Network & Internet → Mobile hotspot → toggle On, "Share over" set to Wi-Fi. Skip this if it's already on.

2 PHONE

Connect to the hotspot

Settings → Wi-Fi → join the laptop's network with its password. Confirm under the network's info that the phone's address starts with `192.168.137.`

3 LAPTOP

Open an elevated terminal

Start → type `powershell` → right-click → **Run as administrator**. Confirm the title bar reads "Administrator: Windows PowerShell" — a regular terminal will fail with a permission error.

```
Administrator: Windows PowerShell

PS> cd C:\NetLens
PS> venv\Scripts\Activate.ps1
```

4 LAPTOP

Start the capture

The hotspot adapter has already been confirmed as `Local Area Connection* 4` (IP `192.168.137.1`) — only re-run `--list-interfaces` / check `ipconfig` if that ever changes.

```
(venv) PS C:\NetLens>

python main.py --iface "Local Area Connection* 4"

# if the adapter number ever changes:
python main.py --list-interfaces
ipconfig   # look for IPv4 192.168.137.x with no default gateway
```

5 PHONE

Generate traffic

Open Safari (or any app) and load a page. Do this *while* watching the laptop terminal — packet rows should scroll in within a second or two.

6 LAPTOP

Let it run, then stop

Wait for at least one "Flows" table to print (every ~50 packets / ~10 seconds). Press `Ctrl+C` to stop — everything captured stays in `storage/netlens.db`.

OPTIONAL: RUN THE TEST SUITE INSTEAD

To show the decoder/flow/storage logic is verified without needing a live phone in the room:

```
(venv) PS C:\NetLens>
pytest tests/ -v   # 19 tests: decoder, flow grouping, SQLite round-trips
```

How this differs from Wireshark

Wireshark isn't the competition — it's the mature tool NetLens is deliberately not trying to be yet.

	NETLENS (PHASE 1)	WIRESHARK
How headers get parsed	By hand, field by field, in code the team wrote and can explain byte-by-byte	A mature dissection engine covering hundreds of protocols
What it proves	That the team actually understands frame/header structure	Nothing about the user's own understanding — it's a finished tool
Protocol coverage	Ethernet, IPv4, TCP, UDP, ARP, DNS	Effectively everything
Conversation view	Built-in flow table, grouped by 5-tuple, persisted to SQLite	"Conversations" view exists but is a secondary feature over a live/loaded capture
Persistence	SQLite database (packets + flows tables), queryable after the fact	.pcap/.pcapng files — raw frames, no built-in structured storage
Interface	Terminal — live feed + periodic flow table	Full GUI — packet list, protocol tree, hex view, display filters
Where it's headed	Baseline learning + anomaly detection rules built directly on this data	Manual analysis — a human interprets what they see

In short: Wireshark is what you reach for when you need to inspect anything, on any network, right now. NetLens is a purpose-built pipeline — narrower in protocol coverage, but opinionated about storage and flow structure in a way that sets up the detection logic coming after Phase 1.

Protocol byte maps

The exact offsets `decoder.py` reads from. This is the reference for the "point at a live packet and derive each field by hand" bar — every team member should be able to work through these without looking at the code.

Ethernet II

14 BYTES FIXED

BYTES	FIELD	NOTES
0–5	Destination MAC	6 bytes
6–11	Source MAC	6 bytes
12–13	EtherType	big-endian uint16 — 0x0800 IPv4, 0x0806 ARP, 0x8100 VLAN tag (real type pushed 4 bytes further in)

IPv4

20 BYTES MINIMUM

BYTES	FIELD	NOTES
0	Version + IHL	top nibble version, bottom nibble header length in 32-bit words
2–3	Total length	uint16
6–7	Flags + fragment offset	top 3 bits flags, low 13 bits offset
8	TTL	
9	Protocol	6 = TCP, 17 = UDP, 1 = ICMP
12–15	Source IP	
16–19	Destination IP	

TCP

20 BYTES MINIMUM

BYTES	FIELD
0–1	Source port
2–3	Destination port
4–7	Sequence number
8–11	Ack number
12	Data offset (top nibble)
13	Flags: FIN,SYN,RST,PSH,ACK,URG

UDP

8 BYTES FIXED

BYTES	FIELD
0–1	Source port
2–3	Destination port
4–5	Length
6–7	Checksum

ARP

28 BYTES FIXED, IPV4-OVER-ETHERNET

BYTES	FIELD	NOTES
6–7	Opcode	1 = request, 2 = reply
8–13	Sender MAC	
14–17	Sender IP	
18–23	Target MAC	
24–27	Target IP	

DNS `12-BYTE HEADER + VARIABLE QUESTION/ANSWERS`

BYTES	FIELD	NOTES
2–3	Flags	top bit of byte 2: 0 = query, 1 = response
4–5	QDCOUNT	number of questions
6–7	ANCOUNT	number of answers
12+	Question / answer records	names are length-prefixed labels ending in a zero byte; a length byte ≥ 0xC0 is a compression pointer back into the message

Definition of done

- [x] Laptop captures live traffic from a phone connected through it as a hotspot
- [x] Ethernet / IP / TCP / UDP headers parsed manually — no scapy dissection shortcuts
- [x] ARP and DNS packets identified with key fields extracted
- [x] Packets grouped into flows by 5-tuple
- [x] Everything stored in SQLite — both `packets` and `flows` tables
- [x] Terminal view shows a live packet feed and a flow table
- [x] Any team member can pick a live packet, point at the raw bytes, and manually derive each printed field — see the byte maps above

Worked example — decode one packet entirely by hand

This is the real 54-byte fixture in `decoder/test_data/tcp_packet.bin` (a TCP SYN, source 10.0.0.5:51000 to destination 93.184.216.34:443), annotated byte by byte. This is exactly what "point at a live packet and derive each field by hand" looks like — walk through this table live if asked.

raw bytes, hex

```
aabbccddee02 aabbccddee01 0800 45000028000100004006 3af0 0a000005 5db8d822 c738 01bb 000003e8 00000000 5002 2000 8327 000
```

BYTES	HEX	LAYER & FIELD	DERIVATION
0–5	`aa bb cc dd ee 02`	Ethernet — destination MAC	Six raw bytes, colon-joined
6–11	`aa bb cc dd ee 01`	Ethernet — source MAC	Six raw bytes, colon-joined
12–13	`08 00`	Ethernet — EtherType	0x0800 → IPv4 follows
14	`45`	IPv4 — version / IHL	top nibble 4 = version; bottom nibble 5 × 4 = 20-byte header
16–17	`00 28`	IPv4 — total length	0x0028 = 40 bytes (20 IP + 20 TCP)
20–21	`00 00`	IPv4 — flags / fragment offset	all zero → not fragmented
22	`40`	IPv4 — TTL	0x40 = 64
23	`06`	IPv4 — protocol	6 = TCP → hand off to the TCP parser next
26–29	`0a 00 00 05`	IPv4 — source IP	10.0.0.5, each byte as decimal, dot-joined
30–33	`5d b8 d8 22`	IPv4 — destination IP	93.184.216.34
34–35	`c7 38`	TCP — source port	0xC738 = 51000
36–37	`01 bb`	TCP — destination port	0x01BB = 443
38–41	`00 00 03 e8`	TCP — sequence number	0x000003E8 = 1000
42–45	`00 00 00 00`	TCP — ack number	0 — no ACK yet, this is the opening SYN
46	`50`	TCP — data offset	top nibble 5 × 4 = 20-byte TCP header, no options
47	`02`	TCP — flags	bit 0x02 set → **SYN** only

14 (Ethernet) + 20 (IPv4, IHL×4) + 20 (TCP, data-offset×4) = 54 bytes exactly, matching the frame length — confirming nothing was mis-sliced. This is what `decode()` returns for this exact frame:

PacketRecord

```
protocol=TCP  src_ip=10.0.0.5  dst_ip=93.184.216.34
src_port=51000  dst_port=443  ttl=64
tcp_flags=SYN  tcp_seq=1000  tcp_ack=0
summary="TCP 10.0.0.5:51000 -> 93.184.216.34:443 [SYN]"
```

Frequently asked questions

Anticipated questions, answered from the actual code and the decisions behind it — not guesses.

Why not just let scapy parse the packets too, since it is already a dependency for capture?

The entire point of Phase 1 is proving the team understands protocol structure well enough to derive it by hand. Scapy's dissection would produce the same output with zero evidence anyone understands why byte 9 of an IPv4 header is the protocol number. Scapy is confined to two jobs that genuinely need it: talking to Npcap for raw frames (`capture.py`), and building syntactically valid frames as test fixtures (`generate_samples.py`). `decoder.py` never imports it.

Why does capture need Administrator on Windows?

Reading raw frames off a NIC bypasses the normal TCP/IP stack — a process doing that can see every frame on the wire, not just traffic addressed to it, which is exactly what makes packet sniffing a privileged, security-sensitive capability. Npcap enforces elevation on Windows for this reason; the Linux/macOS equivalent is root or the `CAP_NET_RAW` capability.

What is the background thread and queue in capture.py actually for?

Scapy's `sniff()` is blocking and owns its own loop — it never hands control back between packets. Running it on a background thread that pushes each frame into a `queue.Queue`, while the main thread pulls with a timeout, decouples "wait for the next frame" from everything else `main.py` needs to do periodically, like checking whether it's time to print a flow table.

Why is a flow only one direction — doesn't that undercount a real conversation?

It's a deliberate, documented Phase 1 simplification. A TCP exchange naturally becomes two related flows here (client→server, server→client) since their 5-tuples have source/destination swapped. Merging both directions into one bidirectional flow needs a rule for which 5-tuple is "canonical" — straightforward, but intentionally deferred so the tracker could ship simple and correct inside the three-day window, instead of simple but wrong.

Why does storing a flow use INSERT OR REPLACE instead of just appending a new row each time?

Flows are recomputed from scratch every refresh — `group_into_flows(get_all_packets(conn))` re-reads and regroups every packet stored so far. That's already a complete, correct snapshot, so appending it as new rows every 10 seconds would just pile up thousands of duplicate snapshots of the same conversation. Upserting on the 5-tuple primary key keeps exactly one current row per flow.

Isn't recomputing every flow from every stored packet on every refresh wasteful?

At Phase 1 scale — a few thousand packets across one demo — it's instant, and it's trivially correct: there's no incremental-update logic that could silently drift from the true state. It's a known, deliberate place to optimize later if packet volume ever makes it matter.

What happens to a malformed or truncated packet from real traffic?

Every parser in `decoder.py` checks length before reading and falls back to an `OTHER`-tagged, non-crashing record if a header doesn't fit — e.g. `_parse_ipv4` checks `len(ip) < IPV4_MIN_HEADER_LEN` before touching a single field. On top of that, `main.py` wraps decode and store in a try/except that logs a one-line warning and keeps the capture loop running, so one bad frame from messy live traffic can never take the whole session down.

Why is DNS parsed separately instead of being its own top-level protocol like TCP or ARP?

DNS isn't a new layer in the Ethernet/IP/TCP/UDP sense — it's an application payload riding inside a UDP or TCP payload on port 53. Once a record is already correctly identified as UDP or TCP, `decoder.py` checks if either port is 53 and, if so, best-effort parses the payload as DNS and layers extra fields (`dns_query_name`, `dns_answers`, …) onto that same record rather than replacing its protocol.

How does DNS name compression work, and what stops it looping forever?

A DNS message often repeats the same domain name in several places. Rather than spell it out twice, RFC 1035 lets a name end early with a 2-byte pointer (any label-length byte ≥ 0xC0) whose low 14 bits are a byte offset back into the message where the name continues. `_parse_dns_name` jumps to that offset — but a corrupted or hostile message could point at itself and loop forever, so it counts jumps and raises past `DNS_MAX_POINTER_JUMPS` (20) instead of hanging.

Why SQLite instead of Postgres or MySQL?

Zero setup — a single file, no server process to install or run, which matters when the build needs to work on any teammate's laptop inside three days. Python's standard library ships `sqlite3`, so it's not even an extra dependency. It comfortably handles one demo session's worth of packets; a production, multi-user deployment would likely outgrow it, but that isn't Phase 1's problem.

Are the SQL queries safe from injection, given the data comes off the network?

Yes — every query in `db.py` uses parameterized placeholders (`?`) with values passed separately to `conn.execute()`. Nothing decoded from a packet,

which is attacker-influenced by definition, is ever string-formatted directly into SQL.

How is this "flow" different from a firewall's or NAT's "session"?

Conceptually related — both key on the 5-tuple — but a stateful firewall session tracks actual TCP state transitions (SYN-SENT, ESTABLISHED, FIN-WAIT, …) and times sessions out. `flow_tracker.py` does neither; it's a pure aggregation (count, bytes, first/last seen) with no concept of a flow "ending." That's sufficient for a live summary and future baseline learning, but it isn't connection-tracking.

Why add --iface / --list-interfaces — wasn't the scope just "capture"?

It came directly out of actually running the demo: Windows creates several near-identical "Local Area Connection* N" virtual adapters, and hotspot traffic only appears on whichever one is actively bound to it. Scapy's default interface has no way to know that. Without a way to list and pick interfaces, capture was "working" in code but not actually usable for this exact setup — a Phase 1 necessity, not scope creep.

What comes right after Phase 1?

Per the plan: enrichment (resolving destination IPs to organisation names), baseline learning (establishing what's normal for this device and network), and three detection rules built on top of that baseline. Their specifics aren't guessed at here since they haven't been designed yet — Phase 1's only job was proving decoded, stored, flow-grouped data exists to build them on.

Why Python for something this low-level, instead of C?

`struct.unpack` format strings (`"!HHIIBB"`, etc.) specify field widths and byte order exactly as precisely as a C struct would — nothing about the RFC-level understanding demonstrated changes with the language. Python trades raw throughput for development speed and readability, the right trade for a three-day proof of foundation.

Troubleshooting log

Three real snags hit while bringing this demo up the first time, and exactly how each was diagnosed.

"net session" says Access is denied inside VS Code's terminal

VS Code itself wasn't launched elevated, so its integrated terminal inherits that. Fix: open a separate PowerShell window via Start → right-click → *Run as administrator*, confirm the title bar reads "Administrator: Windows PowerShell", then re-check with `net session` (an empty list with no error is success).

--list-interfaces shows several near-identical "Local Area Connection* N" entries

The name alone can't tell you which one is the live hotspot. Fix: cross-reference with `ipconfig` and pick the adapter with an IPv4 address in the `192.168.137.x` range (Windows' fixed subnet for Mobile Hotspot/ICS) and a blank default gateway — this machine *is* the gateway on that adapter.

First flow table only shows ARP / DHCP / NetBIOS / LLMNR / IGMP, no real browsing

Normal, not a bug — this is the first ~50 packets: the phone joining the network (DHCP lease, ARP probe) plus the laptop's own background chatter, not application traffic yet. No fix needed; browse on the phone while watching the terminal and the next flow table fills in with TCP/DNS rows to real destinations.

Appendix — full annotated source

The complete, current source for every core module, verbatim — so any question about exact behaviour can be answered by reading the actual code below rather than a paraphrase of it.

```
"""
Shared data contract for NetLens.

Every decoded packet, from any source (live capture, pcap replay, test
fixtures), must be represented as a PacketRecord. capture/ produces raw
bytes, decoder/ turns those bytes into a PacketRecord, and storage/
consumes PacketRecords. No module should invent its own packet shape.

Do not edit without agreement from the whole team - all three modules
depend on this shape matching exactly.
"""

from dataclasses import dataclass, field
from typing import List, Optional


@dataclass
class PacketRecord:
    # --- Capture metadata ---
    timestamp: float  # epoch seconds, from the capture layer
    length: int        # total frame length in bytes, as seen on the wire

    # --- Ethernet (layer 2) - present on every frame ---
    src_mac: str        # e.g. "aa:bb:cc:dd:ee:ff"
    dst_mac: str
    eth_type: int        # EtherType, e.g. 0x0800 = IPv4, 0x0806 = ARP

    # --- Network / transport 5-tuple ---
    # Populated for IPv4 and ARP frames. For ARP, src_ip/dst_ip hold the
    # sender/target protocol addresses so ARP traffic can still be grouped
    # into flows alongside IP traffic. None for anything else (e.g. IPv6,
    # unrecognised EtherTypes) that Day 1 doesn't decode.
    protocol: Optional[str] = None   # "TCP" | "UDP" | "ICMP" | "ARP" | "OTHER"
    src_ip: Optional[str] = None
    dst_ip: Optional[str] = None
    src_port: Optional[int] = None   # TCP/UDP only
    dst_port: Optional[int] = None   # TCP/UDP only

    # --- IPv4 header extras ---
    ttl: Optional[int] = None
    ip_total_length: Optional[int] = None

    # --- TCP extras ---
    tcp_flags: Optional[str] = None   # e.g. "SYN,ACK"
    tcp_seq: Optional[int] = None
    tcp_ack: Optional[int] = None

    # --- ARP extras ---
    arp_opcode: Optional[int] = None       # 1 = request, 2 = reply
    arp_sender_mac: Optional[str] = None
    arp_target_mac: Optional[str] = None

    # --- DNS extras (application layer, riding on UDP/TCP port 53) ---
    dns_is_response: Optional[bool] = None
    dns_query_name: Optional[str] = None
    dns_query_type: Optional[str] = None
```

```
    dns_answers: List[str] = field(default_factory=list)

    # --- Fallback / debugging ---
    summary: Optional[str] = None   # short human-readable one-liner
```

```
"""
Person A's part: get raw bytes off the wire.

capture_packets() does NOT parse anything - it just hands back
(raw_bytes, timestamp) tuples for every frame scapy's sniff() sees on the
active interface. All header parsing lives in decoder/decoder.py.

--- Privilege / driver requirements (read this before running) ---

Windows:
  - Requires Npcap (https://npcap.com/) installed with "Support raw 802.11
    traffic (and monitor mode) for wireless adapters" NOT required, but
    "Install Npcap in WinPcap API-compatible Mode" should be checked so
    scapy can find it.
  - Must be run from a terminal opened "as Administrator". Packet capture
    needs raw access to the NIC driver, which Npcap only grants to
    elevated processes on Windows.
  - If Npcap isn't installed, or the terminal isn't elevated, sniff() will
    raise a PermissionError / OSError, or scapy will report no capture
    devices at all.

Linux/macOS (for reference, not this machine):
  - Requires root (sudo) or the CAP_NET_RAW capability on the interpreter.

This module always uses scapy's sniff() (never raw sockets directly),
since Npcap/libpcap is what actually provides raw-frame access.
"""

from __future__ import annotations

import queue
import threading
from typing import Iterator, Optional, Tuple

from scapy.all import sniff


def capture_packets(
    iface: Optional[str] = None,
    count: int = 0,
) -> Iterator[Tuple[bytes, float]]:
    """
    Yield (raw_bytes, timestamp) for every frame captured on `iface`
    (or scapy's default/active interface if None), as they arrive.

    count=0 means capture indefinitely; stop by breaking out of the
    consuming loop (e.g. Ctrl+C), which closes this generator.

    No parsing happens here - raw_bytes is the untouched frame as scapy
    handed it to us, straight from Npcap/libpcap.
    """
    frame_queue: "queue.Queue[Tuple[bytes, float]]" = queue.Queue()
    stop_event = threading.Event()
    sniff_error: list = []

    def _on_frame(pkt) -> None:
```

```
        # bytes(pkt) serialises the frame exactly as scapy received it -
        # this is the raw wire data, not a parsed/decoded structure.
        frame_queue.put((bytes(pkt), float(pkt.time)))

    def _run_sniff() -> None:
        try:
            sniff(
                iface=iface,
                prn=_on_frame,
                store=False,
                count=count,
                stop_filter=lambda _pkt: stop_event.is_set(),
            )
        except Exception as exc:  # surfaced to the consumer below
            sniff_error.append(exc)
        finally:
            stop_event.set()

    sniff_thread = threading.Thread(target=_run_sniff, daemon=True)
    sniff_thread.start()

    try:
        while not stop_event.is_set() or not frame_queue.empty():
            try:
                yield frame_queue.get(timeout=0.5)
            except queue.Empty:
                continue
    finally:
        # Consumer stopped iterating (e.g. Ctrl+C / break) - ask the
        # sniff thread to wind down; it'll exit on its next packet or
        # timeout since scapy checks stop_filter per-packet.
        stop_event.set()

    if sniff_error:
        raise sniff_error[0]


if __name__ == "__main__":
    print("NetLens capture sanity check - press Ctrl+C to stop.\n")
    frame_count = 0
    try:
        for raw_bytes, timestamp in capture_packets():
            frame_count += 1
            print(f"[{frame_count:>5}] len={len(raw_bytes):>5} bytes  ts={timestamp:.6f}")
    except KeyboardInterrupt:
        print(f"\nStopped. Captured {frame_count} frame(s).")
    except PermissionError:
        print(
            "\nPermission denied opening the capture device.\n"
            "This script needs to be run from an Administrator terminal on "
            "Windows, and Npcap must be installed (https://npcap.com/) with "
            "'Install Npcap in WinPcap API-compatible Mode' enabled."
        )
    except OSError as exc:
        print(
            f"\nCapture failed to start ({exc}).\n"
```

```
            "Most likely causes:\n"
            "  - Npcap is not installed - get it from https://npcap.com/\n"
            "  - The terminal is not running as Administrator\n"
            "  - No active network interface was found"
        )
```

```
"""
Person B's part: turn raw frame bytes into a PacketRecord by hand.

Every header field below is pulled out with struct.unpack / manual byte
slicing at a known offset - no scapy dissection is used for the actual
parsing. Scapy is only used elsewhere in the project (capture, and to
build test fixtures here) to produce raw bytes; this file proves we
understand RFC 894 (Ethernet), RFC 791 (IPv4), RFC 793 (TCP), RFC 768
(UDP), RFC 826 (ARP) and RFC 1035 (DNS) header layouts ourselves.

All multi-byte integer fields in these protocols are big-endian
("network byte order"), so every struct format string below starts
with '!'.

decode() is called on real live traffic (see main.py), so every parser
below is defensive about length: a header that's too short to safely
read is reported as truncated/OTHER instead of raising, since a single
malformed or unusual frame must never take the parser down.
"""

from __future__ import annotations

import struct
from typing import List, Optional, Tuple

from shared.packet_record import PacketRecord

# EtherType values (Ethernet II, bytes 12-13 of the frame)
ETH_TYPE_IPV4 = 0x0800
ETH_TYPE_ARP = 0x0806
ETH_TYPE_IPV6 = 0x86DD
ETH_TYPE_VLAN = 0x8100  # 802.1Q tag - real EtherType is 4 bytes further in

# IPv4 "protocol" field values (byte 9 of the IPv4 header)
IP_PROTO_TCP = 6
IP_PROTO_UDP = 17
IP_PROTO_ICMP = 1

TCP_FLAG_BITS = (
    (0x01, "FIN"),
    (0x02, "SYN"),
    (0x04, "RST"),
    (0x08, "PSH"),
    (0x10, "ACK"),
    (0x20, "URG"),
)

DNS_PORT = 53

ETHERNET_HEADER_LEN = 14
VLAN_TAG_LEN = 4
IPV4_MIN_HEADER_LEN = 20
IPV6_HEADER_LEN = 40
ARP_PACKET_LEN = 28
TCP_MIN_HEADER_LEN = 20
UDP_HEADER_LEN = 8
```

```
DNS_MIN_HEADER_LEN = 12


def _format_mac(raw: bytes) -> str:
    return ":".join(f"{b:02x}" for b in raw)


def _format_ipv4(raw: bytes) -> str:
    return ".".join(str(b) for b in raw)


def _format_ipv6(raw: bytes) -> str:
    # 8 groups of 2 bytes each, printed as hex, colon-separated. Real
    # tooling would also apply the "::" zero-compression shorthand; we
    # skip that here since this is only used for OTHER-bucketed frames.
    groups = struct.unpack("!8H", raw)
    return ":".join(f"{g:x}" for g in groups)


def _parse_ethernet(raw: bytes) -> Optional[Tuple[str, str, int, int]]:
    """
    Ethernet II header - fixed 14 bytes at the start of every frame:
      bytes 0-5   : destination MAC
      bytes 6-11  : source MAC
      bytes 12-13 : EtherType (big-endian uint16)

    If EtherType is 0x8100 (802.1Q VLAN tag), the real EtherType is
    pushed back another 4 bytes: 2 bytes of tag control info we don't
    care about, then the actual EtherType:
      bytes 12-13 : 0x8100 (tag marker)
      bytes 14-15 : VLAN tag control info (priority/VLAN id, unused here)
      bytes 16-17 : real EtherType

    Returns (src_mac, dst_mac, eth_type, header_len) where header_len is
    where the next layer starts (14, or 18 if a VLAN tag was present).
    Returns None if raw is too short to even hold an Ethernet header.
    """
    if len(raw) < ETHERNET_HEADER_LEN:
        return None

    dst_mac = _format_mac(raw[0:6])
    src_mac = _format_mac(raw[6:12])
    (eth_type,) = struct.unpack("!H", raw[12:14])
    header_len = ETHERNET_HEADER_LEN

    if eth_type == ETH_TYPE_VLAN:
        if len(raw) < ETHERNET_HEADER_LEN + VLAN_TAG_LEN:
            return src_mac, dst_mac, eth_type, header_len
        (eth_type,) = struct.unpack("!H", raw[16:18])
        header_len = ETHERNET_HEADER_LEN + VLAN_TAG_LEN

    return src_mac, dst_mac, eth_type, header_len


def _parse_ipv4(raw: bytes, offset: int) -> Optional[dict]:
    """
```

```
    IPv4 header - starts right after the Ethernet (+ optional VLAN tag)
    header. Minimum 20 bytes; IHL (bottom nibble of byte 0) counts in
    32-bit words, so header length in bytes = IHL * 4 (there can be
    options after byte 20, which we skip over using that length).

      byte  0      : version (top nibble) + IHL (bottom nibble)
      byte  1      : DSCP/ECN (unused here)
      bytes 2-3    : total length (uint16)
      bytes 4-5    : identification (unused here)
      bytes 6-7    : flags (top 3 bits) + fragment offset (low 13 bits)
      byte  8      : TTL
      byte  9      : protocol (6=TCP, 17=UDP, 1=ICMP)
      bytes 10-11  : header checksum (unused here)
      bytes 12-15  : source IP
      bytes 16-19  : destination IP

    Non-zero fragment offset means this is a non-initial IP fragment:
    its payload is a raw continuation of the original datagram, NOT a
    fresh TCP/UDP header, so we deliberately don't hand it to the L4
    parsers (that would decode a random offset into the data as if it
    were a port/flags/etc and produce garbage).
    """
    ip = raw[offset:]
    if len(ip) < IPV4_MIN_HEADER_LEN:
        return None

    version_ihl = ip[0]
    ihl_words = version_ihl & 0x0F
    header_len = ihl_words * 4  # bytes, includes any IP options
    if header_len < IPV4_MIN_HEADER_LEN or len(ip) < header_len:
        return None  # malformed IHL / truncated before header even ends

    (total_length,) = struct.unpack("!H", ip[2:4])
    flags_and_frag = struct.unpack("!H", ip[6:8])[0]
    fragment_offset = flags_and_frag & 0x1FFF
    more_fragments = bool(flags_and_frag & 0x2000)
    ttl = ip[8]
    protocol_num = ip[9]
    src_ip = _format_ipv4(ip[12:16])
    dst_ip = _format_ipv4(ip[16:20])

    return {
        "total_length": total_length,
        "ttl": ttl,
        "protocol_num": protocol_num,
        "src_ip": src_ip,
        "dst_ip": dst_ip,
        "is_fragment": fragment_offset != 0 or more_fragments,
        "is_initial_fragment": fragment_offset == 0,
        "payload": ip[header_len:],
    }


def _parse_tcp(segment: bytes) -> Optional[dict]:
    """
    TCP header - minimum 20 bytes.
```

```
      bytes 0-1  : source port (uint16)
      bytes 2-3  : destination port (uint16)
      bytes 4-7  : sequence number (uint32)
      bytes 8-11 : acknowledgment number (uint32)
      byte  12   : data offset (top nibble, in 32-bit words) + reserved
      byte  13   : flags (FIN,SYN,RST,PSH,ACK,URG in the low 6 bits)
      bytes 14-15: window size (unused here)
      bytes 16-17: checksum (unused here)
      bytes 18-19: urgent pointer (unused here)
    """
    if len(segment) < TCP_MIN_HEADER_LEN:
        return None

    src_port, dst_port, seq, ack, data_offset_reserved, flag_byte = struct.unpack(
        "!HHIIBB", segment[0:14]
    )
    data_offset = (data_offset_reserved >> 4) * 4  # header length in bytes
    if data_offset < TCP_MIN_HEADER_LEN or data_offset > len(segment):
        data_offset = TCP_MIN_HEADER_LEN if len(segment) >= TCP_MIN_HEADER_LEN else len(segment)

    flags = [name for bit, name in TCP_FLAG_BITS if flag_byte & bit]

    return {
        "src_port": src_port,
        "dst_port": dst_port,
        "seq": seq,
        "ack": ack,
        "flags": ",".join(flags) if flags else "",
        "payload": segment[data_offset:],
    }


def _parse_udp(segment: bytes) -> Optional[dict]:
    """
    UDP header - fixed 8 bytes.
      bytes 0-1 : source port (uint16)
      bytes 2-3 : destination port (uint16)
      bytes 4-5 : length (header + data, unused here)
      bytes 6-7 : checksum (unused here)
    """
    if len(segment) < UDP_HEADER_LEN:
        return None

    src_port, dst_port, _length, _checksum = struct.unpack("!HHHH", segment[0:8])
    return {
        "src_port": src_port,
        "dst_port": dst_port,
        "payload": segment[8:],
    }


def _parse_arp(raw: bytes, offset: int) -> Optional[dict]:
    """
    ARP packet - starts right after Ethernet (+ optional VLAN tag),
    fixed 28 bytes for IPv4-over-Ethernet ARP (the only kind we handle):
      bytes 0-1   : hardware type (unused here, always 1=Ethernet)
```

```
      bytes 2-3   : protocol type (unused here, always 0x0800=IPv4)
      byte  4     : hardware address length (unused here, always 6)
      byte  5     : protocol address length (unused here, always 4)
      bytes 6-7   : opcode (1=request, 2=reply)
      bytes 8-13  : sender hardware address (MAC)
      bytes 14-17 : sender protocol address (IP)
      bytes 18-23 : target hardware address (MAC)
      bytes 24-27 : target protocol address (IP)
    """
    arp = raw[offset : offset + ARP_PACKET_LEN]
    if len(arp) < ARP_PACKET_LEN:
        return None

    (opcode,) = struct.unpack("!H", arp[6:8])
    sender_mac = _format_mac(arp[8:14])
    sender_ip = _format_ipv4(arp[14:18])
    target_mac = _format_mac(arp[18:24])
    target_ip = _format_ipv4(arp[24:28])

    return {
        "opcode": opcode,
        "sender_mac": sender_mac,
        "sender_ip": sender_ip,
        "target_mac": target_mac,
        "target_ip": target_ip,
    }


DNS_MAX_POINTER_JUMPS = 20  # guards against a malicious/malformed pointer cycle


def _parse_dns_name(data: bytes, offset: int) -> Tuple[str, int]:
    """
    DNS names are a sequence of length-prefixed labels ending in a zero
    byte, e.g. b'\\x07example\\x03com\\x00' -> "example.com".

    A label length byte with its top two bits set (>= 0xC0) is instead a
    *compression pointer*: the remaining 14 bits are an offset back into
    the message where the rest of the name continues (RFC 1035 4.1.4).
    Real-world DNS responses use these constantly, so we follow them.

    Returns (name, offset_after_this_name_in_the_original_buffer).
    Raises ValueError on out-of-range offsets or a pointer chain that
    doesn't terminate - callers treat DNS parsing as best-effort and
    catch this rather than letting it crash the whole packet.
    """
    labels: List[str] = []
    pos = offset
    end_pos: Optional[int] = None  # where reading resumes after pointer jump
    jumps = 0

    while True:
        if pos >= len(data):
            raise ValueError("DNS name ran past end of message")

        length_byte = data[pos]
```

```
        if length_byte == 0:
            pos += 1
            if end_pos is None:
                end_pos = pos
            break

        if length_byte & 0xC0 == 0xC0:
            if pos + 2 > len(data):
                raise ValueError("truncated DNS compression pointer")
            jumps += 1
            if jumps > DNS_MAX_POINTER_JUMPS:
                raise ValueError("too many DNS compression pointer jumps")
            (pointer_word,) = struct.unpack("!H", data[pos : pos + 2])
            pointer = pointer_word & 0x3FFF
            if end_pos is None:
                end_pos = pos + 2
            pos = pointer
            continue

        pos += 1
        if pos + length_byte > len(data):
            raise ValueError("DNS label ran past end of message")
        labels.append(data[pos : pos + length_byte].decode("ascii", errors="replace"))
        pos += length_byte

    return ".".join(labels), end_pos


DNS_QTYPES = {1: "A", 2: "NS", 5: "CNAME", 15: "MX", 16: "TXT", 28: "AAAA"}


def _parse_dns(payload: bytes) -> Optional[dict]:
    """
    DNS message header - fixed 12 bytes:
      bytes 0-1  : transaction ID (unused here)
      bytes 2-3  : flags - bit 15 (top bit of byte 2) is QR: 0=query, 1=response
      bytes 4-5  : QDCOUNT - number of questions
      bytes 6-7  : ANCOUNT - number of answer records
      bytes 8-9  : NSCOUNT (unused here)
      bytes 10-11: ARCOUNT (unused here)

    Followed by the question section: QNAME (variable), then QTYPE
    (uint16) and QCLASS (uint16). We parse the first question and, for
    responses, walk ANCOUNT answer records to collect their names.

    Raises ValueError/struct.error on malformed input; decode() treats
    DNS enrichment as best-effort on top of an already-valid UDP/TCP
    record, so callers catch those and just skip the DNS fields.
    """
    if len(payload) < DNS_MIN_HEADER_LEN:
        return None

    flags, qdcount, ancount = struct.unpack("!HHH", payload[2:8])
    is_response = bool(flags & 0x8000)

    pos = 12
```

```
    query_name = ""
    query_type = None
    if qdcount > 0:
        query_name, pos = _parse_dns_name(payload, pos)
        if pos + 4 <= len(payload):
            (qtype,) = struct.unpack("!H", payload[pos : pos + 2])
            query_type = DNS_QTYPES.get(qtype, str(qtype))
            pos += 4  # QTYPE + QCLASS

    answers: List[str] = []
    for _ in range(ancount):
        if pos >= len(payload):
            break
        name, pos = _parse_dns_name(payload, pos)
        if pos + 10 > len(payload):
            break
        rtype, _rclass, _ttl, rdlength = struct.unpack("!HHIH", payload[pos : pos + 10])
        pos += 10
        if pos + rdlength > len(payload):
            break
        rdata = payload[pos : pos + rdlength]
        if rtype == 1 and len(rdata) == 4:  # A record
            answers.append(f"{name} -> {_format_ipv4(rdata)}")
        else:
            answers.append(f"{name} -> {DNS_QTYPES.get(rtype, rtype)} record")
        pos += rdlength

    return {
        "is_response": is_response,
        "query_name": query_name,
        "query_type": query_type,
        "answers": answers,
    }


def decode(raw_bytes: bytes, timestamp: float) -> PacketRecord:
    """Parse one raw frame into a PacketRecord. See module docstring."""
    eth = _parse_ethernet(raw_bytes)
    if eth is None:
        # Shorter than a minimum Ethernet header - nothing we can safely
        # read. Shouldn't happen from a real NIC, but a snaplen-truncated
        # or corrupt capture could produce this.
        return PacketRecord(
            timestamp=timestamp,
            length=len(raw_bytes),
            src_mac="",
            dst_mac="",
            eth_type=0,
            protocol="OTHER",
            summary=f"Frame too short to parse ({len(raw_bytes)} bytes)",
        )

    src_mac, dst_mac, eth_type, l3_offset = eth

    record = PacketRecord(
        timestamp=timestamp,
```

```
        length=len(raw_bytes),
        src_mac=src_mac,
        dst_mac=dst_mac,
        eth_type=eth_type,
    )

    if eth_type == ETH_TYPE_ARP:
        arp = _parse_arp(raw_bytes, l3_offset)
        if arp is None:
            record.protocol = "OTHER"
            record.summary = "Truncated ARP packet"
            return record
        record.protocol = "ARP"
        record.src_ip = arp["sender_ip"]
        record.dst_ip = arp["target_ip"]
        record.arp_opcode = arp["opcode"]
        record.arp_sender_mac = arp["sender_mac"]
        record.arp_target_mac = arp["target_mac"]
        record.summary = (
            f"ARP {'request' if arp['opcode'] == 1 else 'reply'}: "
            f"who has {arp['target_ip']}? tell {arp['sender_ip']}"
        )
        return record

    if eth_type == ETH_TYPE_IPV6:
        # Day 1's contract only covers IPv4 5-tuples; IPv6 is bucketed as
        # OTHER rather than decoded, but we still surface the addresses
        # in the summary since the fixed 40-byte header makes that easy
        # and it's far more useful than a bare "unhandled" note.
        record.protocol = "OTHER"
        v6 = raw_bytes[l3_offset : l3_offset + IPV6_HEADER_LEN]
        if len(v6) == IPV6_HEADER_LEN:
            record.summary = f"IPv6 {_format_ipv6(v6[8:24])} -> {_format_ipv6(v6[24:40])} (not decoded)"
        else:
            record.summary = "Truncated IPv6 header"
        return record

    if eth_type != ETH_TYPE_IPV4:
        record.protocol = "OTHER"
        record.summary = f"Unhandled EtherType 0x{eth_type:04x}"
        return record

    ip = _parse_ipv4(raw_bytes, l3_offset)
    if ip is None:
        record.protocol = "OTHER"
        record.summary = "Truncated or malformed IPv4 header"
        return record

    record.src_ip = ip["src_ip"]
    record.dst_ip = ip["dst_ip"]
    record.ttl = ip["ttl"]
    record.ip_total_length = ip["total_length"]

    if ip["is_fragment"] and not ip["is_initial_fragment"]:
        # Non-initial fragment: payload is a raw continuation of the
        # original datagram, not a fresh TCP/UDP header - parsing it as
```

```
        # one would read garbage ports/flags out of arbitrary data.
        record.protocol = "OTHER"
        record.summary = f"IP fragment (proto {ip['protocol_num']}) {record.src_ip} -> {record.dst_ip}"
        return record

    if ip["protocol_num"] == IP_PROTO_TCP:
        tcp = _parse_tcp(ip["payload"])
        if tcp is None:
            record.protocol = "OTHER"
            record.summary = f"Truncated TCP segment {record.src_ip} -> {record.dst_ip}"
            return record

        record.protocol = "TCP"
        record.src_port = tcp["src_port"]
        record.dst_port = tcp["dst_port"]
        record.tcp_flags = tcp["flags"]
        record.tcp_seq = tcp["seq"]
        record.tcp_ack = tcp["ack"]
        record.summary = f"TCP {record.src_ip}:{tcp['src_port']} -> {record.dst_ip}:{tcp['dst_port']} [{tcp['flags']}]"

        if DNS_PORT in (tcp["src_port"], tcp["dst_port"]) and len(tcp["payload"]) > 2:
            # DNS-over-TCP is prefixed with a 2-byte message length (RFC 1035
            # 4.2.2); skip it before handing off to the DNS parser.
            try:
                dns = _parse_dns(tcp["payload"][2:])
            except (ValueError, struct.error):
                dns = None  # not actually DNS, or malformed - keep the TCP record as-is
            _apply_dns(record, dns)

    elif ip["protocol_num"] == IP_PROTO_UDP:
        udp = _parse_udp(ip["payload"])
        if udp is None:
            record.protocol = "OTHER"
            record.summary = f"Truncated UDP segment {record.src_ip} -> {record.dst_ip}"
            return record

        record.protocol = "UDP"
        record.src_port = udp["src_port"]
        record.dst_port = udp["dst_port"]
        record.summary = f"UDP {record.src_ip}:{udp['src_port']} -> {record.dst_ip}:{udp['dst_port']}"

        if DNS_PORT in (udp["src_port"], udp["dst_port"]):
            try:
                dns = _parse_dns(udp["payload"])
            except (ValueError, struct.error):
                dns = None
            _apply_dns(record, dns)

    elif ip["protocol_num"] == IP_PROTO_ICMP:
        record.protocol = "ICMP"
        record.summary = f"ICMP {record.src_ip} -> {record.dst_ip}"

    else:
        # Any other IP protocol number (IGMP, GRE, ESP, OSPF, ...) - the
        # contract only models TCP/UDP/ICMP/ARP, so this is OTHER rather
        # than a crash.
```

```
        record.protocol = "OTHER"
        record.summary = f"IP protocol {ip['protocol_num']} {record.src_ip} -> {record.dst_ip}"

    return record


def _apply_dns(record: PacketRecord, dns: Optional[dict]) -> None:
    if dns is None:
        return
    record.dns_is_response = dns["is_response"]
    record.dns_query_name = dns["query_name"]
    record.dns_query_type = dns["query_type"]
    record.dns_answers = dns["answers"]
    kind = "response" if dns["is_response"] else "query"
    record.summary = f"DNS {kind}: {dns['query_name']} ({dns['query_type']})"
    if dns["answers"]:
        record.summary += f" -> {'; '.join(dns['answers'])}"
```

```
"""
Person C's part: group PacketRecords into flows by the 5-tuple
(src_ip, src_port, dst_ip, dst_port, protocol).

Note: this groups strictly by direction - a request (A->B) and its reply
(B->A) land in two different flows, since their 5-tuples differ (src/dst
swapped). That's a deliberate Day 1 simplification; bidirectional flow
merging is integration-day territory, not this module's job today.
"""

from __future__ import annotations

from dataclasses import dataclass, field
from typing import Dict, List, Tuple

from shared.packet_record import PacketRecord

FlowKey = Tuple[str, int, str, int, str]


@dataclass
class FlowStats:
    key: FlowKey
    records: List[PacketRecord] = field(default_factory=list)
    packet_count: int = 0
    byte_count: int = 0
    first_seen: float = 0.0
    last_seen: float = 0.0

    def add(self, record: PacketRecord) -> None:
        self.records.append(record)
        self.packet_count += 1
        self.byte_count += record.length
        if self.packet_count == 1:
            self.first_seen = self.last_seen = record.timestamp
        else:
            self.first_seen = min(self.first_seen, record.timestamp)
            self.last_seen = max(self.last_seen, record.timestamp)


def _flow_key(record: PacketRecord) -> FlowKey:
    return (
        record.src_ip or "",
        record.src_port or 0,
        record.dst_ip or "",
        record.dst_port or 0,
        record.protocol or "OTHER",
    )


def group_into_flows(records: List[PacketRecord]) -> Dict[FlowKey, FlowStats]:
    flows: Dict[FlowKey, FlowStats] = {}
    for record in records:
        key = _flow_key(record)
        if key not in flows:
            flows[key] = FlowStats(key=key)
```

```
        flows[key].add(record)
    return flows


if __name__ == "__main__":
    from storage.sample_records import SAMPLE_RECORDS

    flows = group_into_flows(SAMPLE_RECORDS)
    print(f"Grouped {len(SAMPLE_RECORDS)} records into {len(flows)} flow(s):\n")
    for key, stats in flows.items():
        src_ip, src_port, dst_ip, dst_port, protocol = key
        print(
            f"  [{protocol:>4}] {src_ip}:{src_port} -> {dst_ip}:{dst_port}  "
            f"pkts={stats.packet_count} bytes={stats.byte_count} "
            f"first={stats.first_seen:.3f} last={stats.last_seen:.3f}"
        )
```

storage/db.py

STORAGE — FULL SOURCE

```
"""
Person C's part: persist PacketRecords and flow summaries to SQLite.

The `packets` table mirrors PacketRecord's fields 1:1 so store/retrieve
round-trips lose nothing. dns_answers (a list) is the one field that
doesn't map to a SQLite column type directly - it's stored as a
comma-joined string and split back out on read.

The `flows` table mirrors flow_tracker.FlowStats: one row per 5-tuple
(src_ip, src_port, dst_ip, dst_port, protocol), keyed on that tuple, with
running packet/byte counts and first/last-seen timestamps. It's a
snapshot, not an append-log - store_flows() re-derives flows from every
packet seen so far (see main.py) and upserts each key, so the table
always reflects the latest totals rather than growing per call.
"""

from __future__ import annotations

import sqlite3
from pathlib import Path
from typing import Dict, List, Optional

from shared.packet_record import PacketRecord
from storage.flow_tracker import FlowKey, FlowStats

DEFAULT_DB_PATH = Path(__file__).parent / "netlens.db"

SCHEMA = """
CREATE TABLE IF NOT EXISTS packets (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    timestamp REAL NOT NULL,
    length INTEGER NOT NULL,
    src_mac TEXT NOT NULL,
    dst_mac TEXT NOT NULL,
    eth_type INTEGER NOT NULL,
    protocol TEXT,
    src_ip TEXT,
    dst_ip TEXT,
    src_port INTEGER,
    dst_port INTEGER,
    ttl INTEGER,
    ip_total_length INTEGER,
    tcp_flags TEXT,
    tcp_seq INTEGER,
    tcp_ack INTEGER,
    arp_opcode INTEGER,
    arp_sender_mac TEXT,
    arp_target_mac TEXT,
    dns_is_response INTEGER,
    dns_query_name TEXT,
    dns_query_type TEXT,
    dns_answers TEXT,
    summary TEXT
)
"""
```

```
FLOWS_SCHEMA = """
CREATE TABLE IF NOT EXISTS flows (
    src_ip TEXT NOT NULL,
    src_port INTEGER NOT NULL,
    dst_ip TEXT NOT NULL,
    dst_port INTEGER NOT NULL,
    protocol TEXT NOT NULL,
    packet_count INTEGER NOT NULL,
    byte_count INTEGER NOT NULL,
    first_seen REAL NOT NULL,
    last_seen REAL NOT NULL,
    PRIMARY KEY (src_ip, src_port, dst_ip, dst_port, protocol)
)
"""

_COLUMNS = [
    "timestamp", "length", "src_mac", "dst_mac", "eth_type", "protocol",
    "src_ip", "dst_ip", "src_port", "dst_port", "ttl", "ip_total_length",
    "tcp_flags", "tcp_seq", "tcp_ack", "arp_opcode", "arp_sender_mac",
    "arp_target_mac", "dns_is_response", "dns_query_name", "dns_query_type",
    "dns_answers", "summary",
]

_FLOW_COLUMNS = [
    "src_ip", "src_port", "dst_ip", "dst_port", "protocol",
    "packet_count", "byte_count", "first_seen", "last_seen",
]


def get_connection(db_path: Path = DEFAULT_DB_PATH) -> sqlite3.Connection:
    conn = sqlite3.connect(db_path)
    conn.execute(SCHEMA)
    conn.execute(FLOWS_SCHEMA)
    conn.commit()
    return conn


def store_packet(record: PacketRecord, conn: sqlite3.Connection) -> None:
    placeholders = ", ".join("?" for _ in _COLUMNS)
    column_list = ", ".join(_COLUMNS)

    # Build values in column order, handling the list->str conversion and
    # the bool->int conversion SQLite needs.
    values = []
    for col in _COLUMNS:
        value = getattr(record, col)
        if col == "dns_answers":
            value = ",".join(value) if value else None
        elif col == "dns_is_response" and value is not None:
            value = int(value)
        values.append(value)

    conn.execute(
        f"INSERT INTO packets ({column_list}) VALUES ({placeholders})", values
    )
    conn.commit()
```

```
def get_all_packets(conn: sqlite3.Connection) -> List[PacketRecord]:
    cursor = conn.execute(f"SELECT {', '.join(_COLUMNS)} FROM packets ORDER BY id")
    records = []
    for row in cursor.fetchall():
        kwargs = dict(zip(_COLUMNS, row))
        kwargs["dns_answers"] = kwargs["dns_answers"].split(",") if kwargs["dns_answers"] else []
        kwargs["dns_is_response"] = (
            bool(kwargs["dns_is_response"]) if kwargs["dns_is_response"] is not None else None
        )
        records.append(PacketRecord(**kwargs))
    return records


def store_flows(flows: Dict[FlowKey, FlowStats], conn: sqlite3.Connection) -> None:
    """
    Upsert the current flow snapshot. Callers (see main.py) recompute
    `flows` from every packet stored so far each time, so each key's row
    is replaced with the latest totals rather than accumulated further
    here - the accumulation already happened in group_into_flows().
    """
    placeholders = ", ".join("?" for _ in _FLOW_COLUMNS)
    column_list = ", ".join(_FLOW_COLUMNS)

    for (src_ip, src_port, dst_ip, dst_port, protocol), stats in flows.items():
        conn.execute(
            f"INSERT OR REPLACE INTO flows ({column_list}) VALUES ({placeholders})",
            [
                src_ip, src_port, dst_ip, dst_port, protocol,
                stats.packet_count, stats.byte_count,
                stats.first_seen, stats.last_seen,
            ],
        )
    conn.commit()


def get_all_flows(conn: sqlite3.Connection) -> Dict[FlowKey, FlowStats]:
    """
    Read the persisted flow snapshot back as FlowStats keyed by 5-tuple.
    The returned FlowStats have empty `records` - the flows table stores
    only aggregate counts, not the underlying packets (those live in the
    packets table and can be re-joined by 5-tuple if ever needed).
    """
    cursor = conn.execute(f"SELECT {', '.join(_FLOW_COLUMNS)} FROM flows")
    flows: Dict[FlowKey, FlowStats] = {}
    for src_ip, src_port, dst_ip, dst_port, protocol, packet_count, byte_count, first_seen, last_seen in cursor.fetchall(
        key: FlowKey = (src_ip, src_port, dst_ip, dst_port, protocol)
        flows[key] = FlowStats(
            key=key,
            packet_count=packet_count,
            byte_count=byte_count,
            first_seen=first_seen,
            last_seen=last_seen,
        )
    return flows
```

```
if __name__ == "__main__":
    import tempfile

    from storage.sample_records import SAMPLE_RECORDS

    with tempfile.TemporaryDirectory() as tmp_dir:
        db_path = Path(tmp_dir) / "netlens_test.db"
        conn = get_connection(db_path)

        for record in SAMPLE_RECORDS:
            store_packet(record, conn)

        fetched = get_all_packets(conn)
        conn.close()

        print(f"Stored {len(SAMPLE_RECORDS)} records, fetched {len(fetched)} back.\n")
        for record in fetched:
            print(f"  [{record.protocol:>5}] {record.summary}")
```

storage/display.py

TERMINAL DISPLAY — FULL SOURCE

```
"""
Person C's part: terminal output. print_packet_row() gives one line per
packet; print_flow_table() summarises grouped flows. Both use `rich`.
"""

from __future__ import annotations

from typing import Dict

from rich.console import Console
from rich.table import Table

from shared.packet_record import PacketRecord
from storage.flow_tracker import FlowKey, FlowStats

console = Console()

_PROTOCOL_COLORS = {
    "TCP": "cyan",
    "UDP": "magenta",
    "ARP": "yellow",
    "ICMP": "green",
    "OTHER": "white",
}


def print_packet_row(record: PacketRecord) -> None:
    color = _PROTOCOL_COLORS.get(record.protocol or "OTHER", "white")
    proto = f"[{color}]{record.protocol or '?':<5}[/{color}]"

    if record.src_port is not None or record.dst_port is not None:
        endpoints = f"{record.src_ip}:{record.src_port} -> {record.dst_ip}:{record.dst_port}"
    elif record.src_ip:
        endpoints = f"{record.src_ip} -> {record.dst_ip}"
    else:
        endpoints = f"{record.src_mac} -> {record.dst_mac}"

    extra = f" [{record.tcp_flags}]" if record.tcp_flags else ""
    console.print(
        f"{record.timestamp:>14.3f}  {proto}  {endpoints:<40}{extra}  "
        f"{record.length:>5}B  [dim]{record.summary or ''}[/dim]"
    )


def print_flow_table(flows: Dict[FlowKey, FlowStats]) -> None:
    table = Table(title="Flows", show_lines=False)
    table.add_column("Protocol", style="bold")
    table.add_column("Source")
    table.add_column("Destination")
    table.add_column("Packets", justify="right")
    table.add_column("Bytes", justify="right")
    table.add_column("First seen", justify="right")
    table.add_column("Last seen", justify="right")
    table.add_column("Duration (s)", justify="right")

    for (src_ip, src_port, dst_ip, dst_port, protocol), stats in flows.items():
```

```
        color = _PROTOCOL_COLORS.get(protocol, "white")
        src = f"{src_ip}:{src_port}" if src_port else src_ip
        dst = f"{dst_ip}:{dst_port}" if dst_port else dst_ip
        duration = stats.last_seen - stats.first_seen
        table.add_row(
            f"[{color}]{protocol}[/{color}]",
            src,
            dst,
            str(stats.packet_count),
            str(stats.byte_count),
            f"{stats.first_seen:.3f}",
            f"{stats.last_seen:.3f}",
            f"{duration:.3f}",
        )

    console.print(table)


if __name__ == "__main__":
    from storage.flow_tracker import group_into_flows
    from storage.sample_records import SAMPLE_RECORDS

    console.rule("print_packet_row()")
    for record in SAMPLE_RECORDS:
        print_packet_row(record)

    console.print()
    console.rule("print_flow_table()")
    print_flow_table(group_into_flows(SAMPLE_RECORDS))
```

main.py

INTEGRATION — FULL SOURCE

```
"""
Day 2 integration: wires the three independent modules built on Day 1
into one live pipeline. All the real logic (capturing, decoding,
storing, displaying) stays in capture/, decoder/, and storage/ - this
file only sequences calls between them:

    capture.capture_packets()  ->  decoder.decode()  ->  storage.db.store_packet()
                                                      ->  storage.display.print_packet_row()

Every ~50 packets (or ~10 seconds of traffic, whichever comes first),
it also pulls everything stored so far, groups it into flows, and
prints a flow summary table.

Run:
    python main.py
Stop with Ctrl+C.

See capture/capture.py's module docstring for the Administrator/Npcap
requirements this depends on - if capture can't open the interface,
that same guidance is surfaced below instead of a raw traceback.

To capture a phone's traffic through a Windows mobile hotspot, scapy's
default interface is often the wrong one - the hotspot's clients show up
on a separate virtual adapter (typically named "Local Area Connection*
N", description "Microsoft Wi-Fi Direct Virtual Adapter"). Run with
--list-interfaces to see every adapter Npcap can see, then pass the
right one with --iface "<name>".

Run:
    python main.py --list-interfaces
    python main.py --iface "Local Area Connection* 3"
Stop with Ctrl+C.
"""

from __future__ import annotations

import argparse
import time

from rich.console import Console

from capture.capture import capture_packets
from decoder.decoder import decode
from storage.db import DEFAULT_DB_PATH, get_all_packets, get_connection, store_flows, store_packet
from storage.display import print_flow_table, print_packet_row
from storage.flow_tracker import group_into_flows

console = Console()

FLOW_TABLE_EVERY_N_PACKETS = 50
FLOW_TABLE_EVERY_N_SECONDS = 10



def _parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="NetLens - live packet capture, decode, and flow view.")
    parser.add_argument(
```

```
        "--iface",
        default=None,
        help='Interface name to capture on, e.g. "Local Area Connection* 3" '
        "(the hotspot's virtual adapter). Defaults to scapy's active interface.",
    )
    parser.add_argument(
        "--list-interfaces",
        action="store_true",
        help="List available capture interfaces (name, description, GUID) and exit.",
    )
    return parser.parse_args()


def _list_interfaces() -> None:
    from scapy.arch import get_windows_if_list

    for iface in get_windows_if_list():
        console.print(f"[bold]{iface.get('name')}[/bold]  -  {iface.get('description')}")


def main() -> None:
    args = _parse_args()
    if args.list_interfaces:
        _list_interfaces()
        return

    conn = get_connection()
    console.print(f"[bold]NetLens[/bold] - live capture starting, writing to {DEFAULT_DB_PATH}")
    if args.iface:
        console.print(f"Interface: {args.iface}")
    console.print("Press Ctrl+C to stop.\n")

    packet_count = 0
    warning_count = 0
    last_flow_table_at = time.monotonic()

    try:
        for raw_bytes, timestamp in capture_packets(iface=args.iface):
            try:
                record = decode(raw_bytes, timestamp)
                store_packet(record, conn)
                print_packet_row(record)
                packet_count += 1
            except Exception as exc:
                # One bad frame must never take the whole capture down -
                # log it and keep going.
                warning_count += 1
                console.print(
                    f"[yellow]warning:[/yellow] skipped a {len(raw_bytes)}-byte frame "
                    f"({type(exc).__name__}: {exc})"
                )
                continue

            now = time.monotonic()
            due_by_count = packet_count % FLOW_TABLE_EVERY_N_PACKETS == 0
            due_by_time = now - last_flow_table_at >= FLOW_TABLE_EVERY_N_SECONDS
```

```
            if due_by_count or due_by_time:
                last_flow_table_at = now
                flows = group_into_flows(get_all_packets(conn))
                store_flows(flows, conn)
                console.print()
                print_flow_table(flows)
                console.print()

    except KeyboardInterrupt:
        pass
    except PermissionError:
        console.print(
            "\n[red]Permission denied opening the capture device.[/red]\n"
            "Run this from a terminal opened 'as Administrator' on Windows, "
            "and make sure Npcap is installed (https://npcap.com/) with "
            "'Install Npcap in WinPcap API-compatible Mode' enabled."
        )
        conn.close()
        return
    except OSError as exc:
        console.print(
            f"\n[red]Capture failed to start ({exc}).[/red]\n"
            "Most likely causes: Npcap is not installed, the terminal isn't "
            "running as Administrator, or no active network interface was found."
        )
        conn.close()
        return

    conn.close()
    console.print(f"\nStopped. Processed {packet_count} packet(s), {warning_count} warning(s).")


if __name__ == "__main__":
    main()
```
